

Kanon M2: The Push Aggregator

RedStone Oracle Adaptor, verifying and publishing in one transaction

v1.0

2026-09-30

Equilibrium Group Oy
Meritullinkatu 1
Helsinki
Finland

Logos
RFP-020

TABLE OF CONTENTS

Scope	4
1. What M2 delivered, in one page	5
2. The push aggregator	7
2.1 What it is for	7
2.2 What was built	7
2.3 How it is proven	7
3. Administration	9
3.1 What it is for	9
3.2 What was built, and a decision that changed	9
3.3 How it is proven	10
4. The five feeds	11
4.1 What it is for	11
4.2 What was built	11
4.3 Deployed, and how to check it without trusting this report	12
5. Security	13
5.1 What it is for	13
5.2 The <code>maxAge</code> window, from both ends	13
5.3 What the analysis concludes	14
5.4 The security test set	14
6. What the write costs	16
6.1 What it is for	16
6.2 The answer	16
6.3 The two paths, and an honest gap	16
6.4 Two measurement notes	17
6.5 How it is proven	17
7. Evidence	18
7.1 The suite	18
7.2 End to end	18
7.3 The gates	19
7.4 The matrix, and why it is machine-checked	19
7.5 Where M2 moved the matrix	20
8. What M2 is not	21
8.1 The pull half	21
8.2 M2-03, which could not be built	21
8.3 Other boundaries worth naming	22
9. Open with Logos	23
9.1 Answered since M1, and now narrower	23
9.2 Standing, and re-checked today	23
9.3 One new finding, not yet a question	24
10. Reproducing this	25

Author: Equilibrium Group Oy

Status: Milestone deliverable

Created: 2026-09-30

Label: push-aggregator

Client: Logos

Target: RFP-020 milestone M2, the nineteen tasks M2-00 to M2-02 and M2-04 to M2-19

Scope

M1 built the engine. M2 builds the first vehicle: a SPEL program that takes a signed RedStone payload as instruction data, verifies it with the core M1 delivered, and writes the result into the canonical RFP-019 price account — in one transaction, with no chained call to anything. Around that instruction M2 adds the administration the feeds need, registers the five day-one feeds on a live network, and puts the security claims on a documented footing.

This report keeps M1's shape, because the milestone has the same character: each of sections 2 to 6 states what the piece is for, what was built, and how it is proven, at the level of behaviour rather than code. Section 8 draws the boundary — what M2 is not, and one task that moved out of it — and section 9 states what remains open with Logos.

The organising idea is M1's, unchanged: nothing is claimed here without a test or a CI gate behind it. Every behaviour in sections 2 to 5 is asserted by the suite, every figure in section 6 is generated by the code that asserts it, and the requirement mapping in section 7 is machine-checked, so a claim of completeness cannot quietly outrun its evidence.

Full detail is in the delivered repository: `TRACEABILITY.md` maps every RFP-020 requirement to its implementation and evidence, `adr/` records the thirty-seven design decisions, `COSTS.md` is the cost table in full, and `MANIPULATION-ANALYSIS.md`, `FEEDS.md` and `TESTNET.md` are the three documents M2 adds.

1. What M2 delivered, in one page

The push aggregator is implemented, deployed and priced. `submit_price` takes a RedStone payload as instruction data, verifies it through `verifier-core`, and writes the agreed median into the canonical RFP-019 price account — one instruction, one transaction, no chained call. It has been exercised across a standalone sequencer and on the LEZ public testnet, with all five day-one feeds registered and reading back.

Nineteen of M2's twenty tasks are complete. The twentieth, M2-03, could not be built and is covered in section 8.2.

area	what stands
Push aggregator	<code>submit_price</code> verifies and publishes in a single instruction; per-feed fault isolation; read-only price reads for consumers — section 2
Administration	feed registration, signer-set update, deregistration and an emergency pause, all gated on the RFP-001 admin authority — section 3
Feeds	five feeds registered and exercised on devnet and the public testnet, with the authorised signer rosters recovered rather than trusted — section 4
Security	the manipulation analysis SEC3 asks for, a production <code>maxAge</code> floor and ceiling, and the unauthorised-signer, staleness, replay and input-sanity suites — section 5
Cost	the write measured against the verification it sits beside, and a registration priced — section 6

The numbers: **360 tests, all green**, across seventeen test targets. The write itself costs **7,386 cycles** for a first write and **8,582** for an update, against **3,039,832** to verify the same payload — under three tenths of one percent of the `submit_price` body either way, and 0.0256% of LEZ's 33,554,432-cycle per-transaction budget. The headline of M2 is therefore a negative result, and a useful one: **the write is not where the money goes**. What decides whether a payload fits in a transaction is verification, which M1 already bounded.

Requirement status moved accordingly. Five requirements went from *planned* to *verified* — F6, F7, R1, R2 and R3 — and three more from *planned* to *partial*. Across the whole RFP-020 inventory the tally is now **6 verified, 16 partial, 14 planned**, against 1, 13 and 22 at M1.

Eight new architecture decision records document the design, taking the total to thirty-seven. Seven questions are open with Logos (section 9), every one of them re-checked against its upstream repository on 2026-09-30; one has been answered upstream since M1 and is narrowed accordingly.

What M2 is not: the push half is shipping, the pull half is not. `pull-lib`, the SDK, the relayer and the reference consumers remain the placeholders M1 declared them. Section 8 draws that boundary.

2. The push aggregator

2.1 What it is for

F1 asks for a public-mode push aggregator accepting signed RedStone data packages. P1 adds the constraint that decides the shape: the whole thing must fit in one transaction. M1 delivered the verification; M2 delivers the instruction that calls it and the account write that follows, and shows the two together inside a real transaction rather than a test harness.

2.2 What was built

`submit_price` is one instruction. It takes the RedStone payload as instruction data, the feed account it was registered against, the pinned LEZ clock account and the price account, and it does three things in order: verify the payload against the feed's configuration, decide what the price account should now hold, and hand LEZ the resulting account states. There is no chained call to a second program and no second transaction.

Submission is permissionless, and this is a decision rather than an omission. The transaction's sender is never consulted. Authenticity comes from the registered RedStone signer set and from nowhere else, because the sender attests to nothing about the bytes it carries: a payload is either signed by enough authorised keys or it is not, whoever relays it. The consequence for availability is analysed in section 5, not assumed away.

The instruction has two paths, and the difference matters for cost (section 6). A first submission builds the price account and claims it at a derived address; an update decodes the account already published and moves only what a newer observation is entitled to move. ADR 32 records that one price account exists per feed, at an address anyone can derive, and that anyone may fill it.

Two supporting pieces complete the surface. **Read-only price reads** (M2-04) let a consumer take the published price without a write path. **Per-feed fault isolation** (M2-05) means a submission writes only its own feed's price account, so a failure on one feed cannot reach another — asserted directly by `a_submission_writes_only_its_own_feeds_price_account`.

2.3 How it is proven

Twenty-two tests cover the submission path in `aggregator-program/tests/submit_price.rs` alone, within fifty-two across the aggregator's integration suites. The ones that carry the argument:

claim	test
A payload creates the account	a_captured_payload_creates_the_price_account_on_the_first_submission
The address is the one a client derives	the_claimed_seed_is_the_address_a_client_derives, the_derived_price_account_address_has_not_moved
An update moves only what it may	an_update_moves_price_and_timestamp_and_nothing_else
Replay cannot move the price	replaying_the_same_payload_cannot_move_the_price
Unauthorised signers publish nothing	a_signer_outside_the_registered_set_cannot_publish
The staleness window is two-sided	the_window_is_two_sided_and_both_edges_are_inclusive, a_clock_past_the_windows_upper_edge_refuses_a_captured_payload
One feed cannot touch another's account	a_submission_writes_only_its_own_feeds_price_account
Every failure names its own cause	every_upstream_failure_refuses_with_its_own_cause_and_writes_nothing

That last one is worth drawing out. `SpelError` carries a number and a string, so without deliberate dispatch a caller would see thirteen distinct verification failures arrive as one indistinguishable error. Each leaf cause keeps its own code, because a caller that cannot tell two failures apart cannot act on either — and the test asserts both halves: the cause is preserved, **and** nothing is written.

The evidence that it is one transaction is an executed transaction, not a type. M2-19 runs one `SubmitPrice` transaction carrying the payload as its instruction data across a standalone sequencer; it verifies and publishes, and an update at a later round does the same. “Single transaction” is therefore a statement about transactions that ran, not about the shape of the code. The end-to-end suite lives in its own workspace for the reason ADR [M2-19:01] records.

3. Administration

3.1 What it is for

F6 asks for an admin authority that can register feeds, update a feed's signer set on roster changes, and deregister feeds. SEC2 adds the constraint that the stored signer set be updatable **only** by that authority, with the update path itself tested. The RFP also asks for an emergency pause.

The signer set is the security parameter of a feed. Everything in this section exists so that exactly one party can change it, and so that a roster rotation is an ordinary operation rather than an incident.

3.2 What was built, and a decision that changed

Six instructions: `Initialize`, `RegisterFeed`, `UpdateSignerSet`, `DeregisterFeed`, `PauseFeed` and `UnpauseFeed`. Every one but `SubmitPrice` is gated on the admin authority.

The authority is a local implementation, not RFP-001's, and that is a change from M1.

ADR 14 chose RFP-001's authority through `logos-co/spel` PR 212, designing against its `#[require_admin(config)]` and `AdminConfig`, and held a local shim in reserve on the reasoning that the shim implements the same surface so the gated instructions are written once either way. Three months on, the contingency is the live branch. PR 212 has had no activity since 2026-05-20, while SPEL has shipped v0.6.0 and v0.7.0 and moved to LEZ v0.2.4 around it.

Two properties of 212 also turned out to make it less of a like-for-like substitution than ADR 14 assumed. Its macro reads `AdminConfig` **by name**, so a program embedding the admin state in its own state — which the library's own README offers as an option — cannot use the macro at all. And its sample instructions each build a post-state and then return the account unmodified, so nothing they mutate is persisted, with tests that assert `is_ok()` and therefore never reach the discrepancy. ADR [M2-06:01] records this and supersedes ADR 14 on the shape of the shim, though not on the choice of authority: when RFP-001's interface lands, this is the piece that moves onto it, and section 9 keeps the question open with Logos.

The authority is established by `Initialize`, gated on a genesis key compiled into the guest. Handover is deliberately two transactions — a nomination and an acceptance — so the authority cannot be moved to an address that cannot act, and revocation leaves nobody able to administer. The reasoning, including why a compile-time genesis key rather than first-caller-wins, is in ADR [M2-06:01].

3.3 How it is proven

claim	test
Initialisation establishes a usable authority	<code>an_initialisation_lands_and_claims_the_address_the_constraint_checks,</code> <code>the_authority_an_initialisation_writes_can_administer_a_feed</code>
Handover is two-step and moves once	<code>a_handover_lands_in_two_transactions_and_moves_the_authority_once</code>
Revocation is final	<code>a_revocation_lands_and_leaves_nobody_able_to_administer</code>
An unusable authority cannot be installed	<code>a_pristine_admin_cannot_establish_the_authority,</code> <code>a_pristine_nominee_cannot_accept_the_authority</code>
A rotation takes effect	<code>a_rotation_lands_on_a_feed_that_was_registered,</code> <code>the_rotated_feed_verifies_against_the_new_set</code>
A rotation is scoped to its feed	<code>rotating_one_feed_leaves_another_alone,</code> <code>a_registered_feed_cannot_be_rotated_under_another_id</code>
A pause stops submissions	<code>a_paused_feed_is_refused_by_the_submission_path</code>
Retirement is not a back door	<code>a_retired_feed_refuses_submissions_as_retired,</code> <code>an_emptied_account_owned_by_somebody_else_is_not_re_registerable</code>
A published pair is immutable	<code>a_feed_id_may_not_change_its_pair_once_it_has_published</code>

Two of these deserve a sentence. `a_feed_id_may_not_change_its_pair_once_it_has_published` closes a repointing attack: once a feed id has published a price for a pair, re-registration may not quietly change which pair that id means, because consumers have already written the id down. `an_emptied_account_owned_by_somebody_else_is_not_re_registerable` closes the mirror case on the account side.

F6 is **verified** on this evidence. SEC2 remains **partial**, because its second obligation is the pull side — a consumer's signer set must never be sourced from the payload — and the pull library is M3.

4. The five feeds

4.1 What it is for

F7 names five feeds — BTC/USD, ETH/USD, SOL/USD, XMR/USD and ZEC/USD — and asks for them registered on LEZ devnet or testnet as part of the deliverable. S1 asks for the deployment itself.

Three of the five are safe to assume. The two privacy assets are not, which is why M2-00 went and looked before M2-11 registered anything.

4.2 What was built

All five are published on `redstone-primary-prod`, the data service the M1 conformance vectors were captured from, and a new round lands about every ten seconds. The two privacy feeds are there.

The authorised signer roster was recovered, not read off a label. For every package captured, the signer address was recovered from the signature over the reconstructed package and compared against the address the gateway served beside it. Across 2,800 packages the two agreed every time, and every package reconstructed. The distinction is the security point: a roster taken from the gateway's own label is a claim by whoever answered the request, and a wrong roster either locks a feed out or lets a stranger in.

All five feeds share one roster of five addresses. That is not a coincidence of F7's five — sweeping every feed the data service serves, 878 of them, found exactly one roster with no exception anywhere. So a shared roster is a property of `redstone-primary-prod` rather than something these five happen to have in common. ADR 30 nevertheless keeps the signer set **per feed**, because a property of the data service today is not a constraint worth compiling in.

The capture is reproducible and fails honestly:

```
python3 scripts/capture-signer-sets.py --minutes 5
```

It reports a feed as published only for rounds it actually observed, and exits non-zero if any of the five produced none or if no request was answered. A capture that cannot reach the gateway therefore fails rather than reporting five feeds it never saw.

4.3 Deployed, and how to check it without trusting this report

All five feeds are registered on the LEZ public testnet at <https://testnet.lez.logos.co>, recorded 2026-09-11. `TESTNET.md` in the delivered repository carries the program image id, the admin account, the admin config account and the five feed account addresses.

Each feed account was read back after registration and decodes completely — 211 of 211 bytes — as a `FeedAccount` holding its own feed id, eight decimals, a 300,000 ms `max_age`, five signers at a threshold of three, unpaused. All five are distinct accounts, and the config account decodes as an `AdminAccount` naming the admin above with no pending nomination.

Reading the accounts back is the evidence, rather than the registration transactions succeeding. A transaction that returns `is_ok()` proves less than an account that decodes to what was intended, which is the same discipline section 3 found missing in 212's own sample.

F7 is **verified** on this basis. S1 is **partial**: the testnet deployment is the M2 half, and the remaining half is M5's — the full suite plus a testnet smoke green, on the testnet current at that point.

5. Security

5.1 What it is for

SEC3 asks for two things: a minimum recommended `maxAge`, documented for both modes, and a manipulation analysis covering signer compromise, replay of stale packages, and signer-set update delays. S3 asks for the security test set. `MANIPULATION-ANALYSIS.md` in the delivered repository is the document; this section states what it concludes and where it declines to conclude.

5.2 The `maxAge` window, from both ends

The ceiling is fifteen minutes and it is enforced, not advised. A `maxAge` above 900,000 ms is looser than RedStone’s own default, so it would admit packages the upstream default drops. Registration refuses such a configuration. ADR 29 records why this has to be an invariant rather than guidance: `freshness` saturates, so a very large `maxAge` puts the lower edge of the window at zero. Every timestamp in the past then passes — and the feed still looks configured like any other. The failure is silent, which is what makes advice insufficient.

The floor is the sum of three latencies, and only one of them is measured:

term	what it is	measured
Publish interval	the package must exist before anyone can submit it	yes
Submission latency	fetch, assemble, submit, and wait for execution — the clock is read inside the program, so a package ages until execution and not until submission	no, needs M4
Clock skew	a RedStone signer’s clock behind the sequencer makes a package look older than it is	no

M2-00 sampled `redstone-primary-prod`: rounds land every 10,000 ms nominally, and the worst gap observed was 20,000 ms, one round skipped. The figure the floor needs is the worst gap, not the cadence — and a five-minute sample bounds that only from below, so it is a lower bound and is stated as one.

> **Provisional recommendation: a minimum of 120,000 ms (two minutes) for push.**

We state plainly that this is a starting point and not a derivation. One of its three terms is measured and two are not, and the measured one is the small one: 20,000 ms of the 120,000, leaving about 100 seconds for submission latency and clock skew together. Smaller risks refusing honest updates; larger widens the replay window analysed below. The relayer in M4 supplies the second term, and the figure should be revisited then.

The three-minute **forward** tolerance is RedStone's own default and records what they judged adequate for skew. It is not a limit anyone enforces on a signer, and we deliberately do not treat it as one — that would repeat, on the other side of the window, the mistake this document corrects about the fifteen minutes.

5.3 What the analysis concludes

Four findings are worth surfacing here.

Signer compromise: the threshold is a median, so half of it is enough. An attacker holding a bare majority of the threshold can move the agreed price, because the median is decided by the middle report. A higher threshold costs linearly in cycles (section 6) and buys proportionally more resistance; the analysis states the trade rather than recommending a number blind.

Replay has two cases and only one of them is closed. A package replayed into another round is refused. A whole round replayed intact, within the staleness window, is **accepted** — and that is the real replay case, bounded by `maxAge` and by nothing else. This is why the floor above is a security parameter and not just an availability one.

A payload can be denied by an appended package, at no cost to the attacker. Recorded rather than fixed, and its cost bound is what ADR 25 and ADR 26 exist for.

Push and pull are not equal, and pull is weaker. A pull consumer keeps its own signer set fresh because it bypasses the centrally administered one, so a roster rotation reaches push immediately and pull only when the consumer acts. The document says so at the point it matters rather than in a caveat at the end.

MANIPULATION-ANALYSIS.md also carries an explicit **what this document does not cover** section. Two of its conclusions depend on decisions M3 has not made, and it says so at each place rather than once.

5.4 The security test set

task	what it asserts
M2-13	an unauthorised signer is refused even when authorised packages already meet quorum
M2-14	only the admin authority can update a stored signer set, and the update path itself is exercised
M2-15	staleness at both edges, replay, and the update window
M2-16	input sanity, including payloads nobody captured

M2-16 needed payloads that RedStone never published — malformed, adversarial, or simply absent from any capture. ADR [M2-16:01] records where those come from and why generating

them is sound, which matters because a fixture nobody signed is only evidence if its construction is principled.

SEC1 and SEC3 remain **partial**: both have a pull-side obligation that M3 carries. SEC2 is partial for the reason section 3 gives.

6. What the write costs

6.1 What it is for

P1 asks that an update fit in one transaction; P2 asks that every published figure be reproducible from the test suite. M1 priced verification. M2-18 prices the other half — the account write — so that the two together account for the whole instruction.

6.2 The answer

The write is not where the money goes. Both columns are the same payload at five signers, so the verification figure here is the five-signer one rather than M1's three-signer headline of 1,831,638.

	first write	update
verification	3,039,832	3,039,832
the write	7,386	8,582
whole <code>submit_price</code> body	3,047,218	3,048,414
<i>the write, as a share of the body</i>	0.242%	0.282%
<i>the write, against the per-transaction budget</i>	0.0220%	0.0256%

Against LEZ's 33,554,432-cycle per-transaction budget, the write is two and a half hundredths of one percent. P1's "single transaction" is therefore comfortable rather than tight, and it is **verification** that decides whether a payload fits — which is what ADR 25 and ADR 26 already bound, and what makes the maximum payload size the number that matters rather than the write.

The share is computed against 33,554,432 and not against a rounded "32M". An earlier draft used the round number and reported 0.027%; the figures here come from the same constant the assertions use. Registering a feed costs 5,270 cycles, 0.0157% of the budget.

6.3 The two paths, and an honest gap

The two figures differ because `submit_price` has two paths: a first write builds the price account, an update decodes the published one first. The 1,196-cycle difference is **a net between two paths, not a component** — an update pays for a read and for `publish`'s checks, a first write for a claim and for building and encoding the account.

Each of those four is measured on its own and pinned:

measured in isolation	cycles
published-account read, update only	1,442
publish's checks, update only (includes the read above)	2,381
AutoClaim::pda_from_seeds, first write only	1,304
publish::price_account and encoding the result, first write only	422

They do not sum to the gap, and we say so rather than presenting a tidy decomposition: the four-term estimate comes out at 655 against a gap of 1,196, about half way. Isolated calls do not cost what inlined ones do. Publishing the four alongside the discrepancy is more useful than publishing a reconciled number that would be an artefact of the arithmetic.

The recurring cost is the update's, since that is what every heartbeat runs.

6.4 Two measurement notes

Measuring a value without charging for it. The build figure is build-and-encode, observed through `core::hint::black_box`. Three earlier attempts were wrong in instructive ways: observing `built.timestamp` gave 43 cycles and was identical with the call deleted, because the construction was elided entirely; one byte per field gave 51, with the 32-byte copies still elided; a fold over the encoded bytes gave 1,782, of which about 1,360 was the fold itself — real work, but the real path does not do it. `black_box` lets the value escape without generating work, and gives 422. `COSTS.md` carries this table because a measurement that cannot be elided is not obviously distinguishable from one that was.

The body is not the whole instruction. These are figures for the `submit_price` body. SPEL's generated validator derives the price account's address before the body is entered, 1,722 cycles measured separately, and the dispatcher and `SpelOutput` wrapping are outside every figure because reaching them needs the macro's entry point rather than a function call. Stating the boundary is the point: a figure whose scope is unstated is not reproducible.

6.5 How it is proven

Every figure here is generated by the code that asserts it. `COSTS.md` is written by the test run rather than transcribed, so a published figure and an asserted figure cannot drift apart, and CI fails if they do. Section 10 gives the command.

P1, P2 and P3 all remain **partial**, because each has a pull-side figure that M3 carries.

7. Evidence

7.1 The suite

360 tests, all green, across seventeen test targets, plus property tests that generate their inputs rather than choose them. The figure is the sum reported by `cargo test --workspace --locked`; section 10 gives the command.

The aggregator's own integration suites account for fifty-two of these:

suite	tests
submit_price.rs	22
admin.rs	7
deregister_feed.rs	6
register_feed.rs	5
update_signer_set.rs	5
idl.rs	4
instruction_parity.rs	3

7.2 End to end

The push path is exercised across a standalone LEZ sequencer, booted in CI from the same LEZ revision the product lockfiles resolve. Three end-to-end tests: the sequencer is there and producing blocks; the push path verifies and publishes across it; and the five feeds register on the public testnet.

The second of those is the one that converts P1 from a claim about types into a claim about executed transactions. One `SubmitPrice` transaction, carrying the payload as instruction data, verifies and publishes; an update at a later round does the same.

The end-to-end tests resolve in their own workspace, for the reason ADR [M2-19:01] records. ADR 34 records the related change that the sequencer now runs in a container like the node beside it — agreed in principle when the feed registration landed, and deliberately outside the RFP's task list, so it is named here rather than counted as a deliverable.

7.3 The gates

job	what it holds
lint	fmt --check and clippy -D warnings
build-test	the workspace builds and the suite passes, locked
no-std	verifier-core still builds without std
guest	the guest workspaces build as program ids depend on
licenses	the dual-licence allowlist, with three clarifications rather than a widened allowlist
traceability	the requirement matrix matches its sources
sequencer	a Bedrock node and a sequencer boot and serve
guardrails	the published cycle figures, by exact equality

7.4 The matrix, and why it is machine-checked

TRACEABILITY.md maps every RFP-020 requirement to its implementation and its evidence. It is **generated** from `traceability/matrix.toml` and `traceability/requirements.toml`, and `cargo test -p traceability` fails when the committed file and its sources disagree. CI does not regenerate it, deliberately: rendering in CI would let the readable matrix change without anyone reviewing the change.

Ten tests hold the matrix itself, including that every requirement has exactly one row, that every task written anywhere is owned by a row, that every piece of evidence resolves, and that the inventory matches the RFP's own taxonomy. Two further tests hold the boundary between harness-only and product dependencies.

This is what stops a claim of completeness outrunning the evidence: a requirement cannot go missing, and a cited test cannot stop existing, without CI going red.

7.5 Where M2 moved the matrix

requirement	M1 → M2	what closed it
F6	planned → verified	the admin authority and its four instructions, section 3
F7	planned → verified	five feeds registered and read back, section 4
R1	planned → verified	per-feed fault isolation
R2	planned → verified	the admin-gated update path
R3	planned → verified	the emergency pause
S1	planned → partial	testnet deployment; the M5 half remains
SEC2	planned → partial	the push side; pull is M3
U4	planned → partial	the aggregator IDL as a published surface

Across the whole inventory of thirty-six requirements the tally moved from 1 verified, 13 partial and 22 planned at M1, to **6 verified, 16 partial and 14 planned** at M2.

The large number still *partial* is structural rather than a shortfall: most of those requirements have a pull-mode obligation, and pull is M3. A requirement is only marked verified when **every** clause of it has evidence, which is why the push side landing does not move a two-mode requirement out of partial.

8. What M2 is not

8.1 The pull half

M2 ships the push mode. The pull mode is M3, and the repository makes that visible: `pull-lib`, `kanon-sdk`, `kanon-relayer`, `kanon-app` and `reference-consumers` are the placeholders M1 declared, each naming the milestone that fills it. They are in the tree so the shape of the finished system is legible, not because they contain work.

This is why so many requirements remain *partial* (section 7.5). The verification core is shared — that was M1's whole point — but a requirement that names both modes needs both modes before it is verified.

8.2 M2-03, which could not be built

One M2 task is not delivered: M2-03, structured `lez-events` emission. It was scheduled, and it is not done. The reason is worth stating precisely, because it is not a matter of effort.

LEZ had no events at **v0.2.0**, which is the version this deliverable pins throughout — `lee_core`, `clock_core`, `lee`, `common` and `sequencer_service_rpc` all resolve to `tag = "v0.2.0"`. There was no emit API for a program to call and no indexer-side type to receive it. The machinery arrived in LEZ **v0.3.0**, in ten pull requests headed by `feat!: events` and `feat!(lee): event selectors`, both flagged as breaking, alongside event rederivation, event filters, an events RPC and FFI exposure.

LEZ v0.3.0 was published on **2026-09-30**, the date of this report. M2's work concluded on 2026-09-11. The task was therefore not startable during M2 by construction, rather than deprioritised within it.

Two consequences follow, and we would rather state both than present the task as merely outstanding:

- **No RFP-020 requirement is left unmet by its absence.** The requirement inventory asks for structured logging in exactly one place — F8, which is the **relayer's** obligation and belongs to M4. Nothing in the RFP asks the on-chain program to emit events. The traceability matrix is complete without M2-03, and no row cites it.
- **It cannot precede the v0.3 migration.** Adopting events means adopting a release with two breaking changes in the path this program sits on. That migration is its own piece of work, tracked as M5-00, and M2-03 now sits behind it rather than in M2.

We have moved M2-03 to M5 accordingly, and corrected our own internal record, which had wrongly marked it complete.

8.3 Other boundaries worth naming

The relay does not exist yet. Section 5 gives a provisional `maxAge` floor of two minutes, and two of its three terms cannot be measured until M4 supplies a relay. The figure should be revisited then, and we would rather ship it labelled provisional than present a derivation we cannot support.

`τgs` deployment is untried. Deploying a program through `τgs` has not been exercised; it is M4's, not M2's.

Cross-machine build reproducibility is not claimed. The guest image id reproduces bit for bit on one machine at the pinned toolchain, including from a cleaned guest tree, and that is the case the deployment story needs. Whether two machines at the same pins produce the same id is untested here and nothing in the repository establishes it. It is recorded as an open question (section 9) rather than as a caveat, because CI builds the guest on GitHub's runners and a release built there and verified locally is precisely the case one machine cannot speak for.

9. Open with Logos

Seven questions are open. **Every one was re-checked against its upstream repository on 2026-09-30**, immediately before this report was sent, and the findings below are that check rather than a restatement of M1's. One question M1 asked has been answered upstream since, and is narrowed accordingly.

This re-check is a standing discipline rather than a courtesy. M1 v1.0 shipped asking Logos about a pull request that had merged a month earlier, and framing an ask around work that had already moved. An ask upstream has already answered is worse than no ask.

9.1 Answered since M1, and now narrower

1. Is a SPEL release coming, or is pinning `main` sanctioned? — answered. SPEL released **v0.7.0 on 2026-09-17**, following v0.6.0. Releases are happening, so pinning `main` is not required and this ask is withdrawn in its original form.

It leaves a narrower one. The product pins SPEL v0.6.0, which resolves LEZ v0.2.0; **v0.7.0 moves that tag to LEZ v0.2.4**. ADR 31 records that a SPEL bump and a LEZ bump are one decision and not two, because cargo unifies the two spellings of the LEZ crates only when source and tag match exactly. So the question to Logos is no longer whether releases exist, but **which release the estate wants us on**, and whether that move belongs with the v0.3 migration (M5-00) rather than on its own. We have not moved the pin unilaterally, because it changes the program image id and therefore every derived account address.

9.2 Standing, and re-checked today

2. Where is the RFP-001 admin-authority extension going to live? `logos-co/spel` PR 212 is **still open and has had no activity since 2026-05-20** — four months. As section 3 explains, the contingency shim is now the live path, and the interface 212 offers turned out not to be the like-for-like substitution ADR 14 assumed. We need to know whether 212 is the intended home, whether the extension crate in a personal namespace is, or whether something else is, so the admin gating can move onto it rather than continue to diverge.

3. Three Logos crates ship without licence metadata. `spel-framework-core` and `spel-framework-macros` declare no `license` field at **v0.7.0**, and the `twap_oracle` manifest on `lez-programs` `main` declares none either. Re-checked 2026-09-30: still missing in all three. They are not unlicensed — `lez-programs` ships a MIT LICENSE and `spel` ships both MIT and Apache-2.0 — but cargo-deny cannot see that. `deny.toml` carries three `[[licenses.clarify]]` entries rather than a widened allowlist, because a clarification names one crate and can be checked. **This remains the cheapest ask on the list:** a one-line `license = "..."` in each manifest removes all three entries and spares every downstream consumer making the same private judgement.

4. May a program claim a non-default, default-owned account? One rule pair, met twice. It strands a signer a program declines to claim, and it lets anyone put one unit of balance on a publicly derivable PDA and make that address permanently unwritable — including the admin config account, whose address takes no input an attacker cannot predict. ADR [M2-06:01] and ADR 33 record the consequences we designed around.

5. Can the canonical price account be made cheaper to depend on? The account-type crate pulls `risc0` and `uniswap_v3_math` in to carry six Borsh fields, and `risc0-zkvm` is pinned at `=3.0.5` — **still an exact pin on `lez-programs` main as of 2026-09-30**. This is closer to a defect report than a preference: relaxing the pin, or splitting out a standalone account-type crate, would make the canonical account usable by the external adaptors it was written for. Kanon is one of those adaptors.

6. What happens to a price account when the program that created it is upgraded?

Recorded in ADR 22 and open on purpose. Under LEZ's ownership model a program id is the hash of its binary, an account is bound for good to the id that created it, and no operation hands an account to a new binary. So an upgraded aggregator cannot update the price accounts the old one created, and consumers would have to follow it to new addresses. Two shapes could answer it: consumers discover the account rather than pin it, or a small permanent program owns the accounts and the upgradeable one writes through it — the only arrangement that keeps the address fixed. Nothing LEZ documents answers it, and **we want the shape settled before M3 pours concrete around one of them**.

7. Is a guest ELF bit-identical across machines at the pinned toolchain? Measured here on one machine, reproducing bit for bit including from a cleaned guest tree. Whether two machines at the same pins agree is not established, and ADR 8 does not settle it — it pins `r0vm` and the guest `rustc` as necessary conditions for the measured **figures**, and records a case where a different ELF gave identical cycle counts, so it is evidence about counts rather than about bytes. It matters because CI builds the guest on GitHub's runners, and verifying a release locally is exactly the case one machine cannot speak for.

9.3 One new finding, not yet a question

LEZ v0.3.0 was published on 2026-09-30, carrying two breaking changes in this program's path — `feat!: events` and `feat!(lee): event selectors` — together with the indexer-side event machinery. This is the release M2-03 was waiting for (section 8.2), and it is also the migration M5-00 exists to carry. We raise it here so the sequencing is visible rather than discovered: adopting it moves the image id, and therefore every derived account address, including those recorded in `TESTNET.md`. We would like to agree the timing rather than pick it.

10. Reproducing this

From the repository root, inside `nix-shell`:

```
cargo test --workspace --locked           # the 360 product tests, minutes
cargo test --release --locked -p kanon-methods # the cost table, built in the guest
cargo test -p traceability                 # the requirement matrix
cargo run -p traceability --bin render-traceability # regenerate TRACEABILITY.md
python3 scripts/capture-signer-sets.py --minutes 5 # re-capture the signer rosters
```

The end-to-end tests resolve in their own workspace (ADR [M2-19:01]) and need a sequencer; `scripts/lez-sequencer.sh` fetches, starts and smoke-tests one, which is what CI does.

Everything section 7.3 describes runs on every push: `ci.yml` carries the lint, build-test, `no_std`, guest, licence, traceability and sequencer jobs; `guardrails.yml` re-asserts M0's frozen figures and the per-component cost table; `lgs-build.yml` proves the toolchain path; `lez-sequencer-image.yml` publishes the prebuilt sequencer image CI boots.

document	contents
TRACEABILITY.md	every RFP-020 requirement, its status, its tasks and its evidence — the generated, machine-checked map of what is done and what remains
COSTS.md	the full cost table, its method, and how to regenerate it
MANIPULATION-ANALYSIS.md	the SEC3 analysis, the <code>maxAge</code> floor and ceiling, and what the analysis does not cover
FEEDS.md	the five feeds, their recovered signer rosters, publish cadence, and how to re-capture all of it
TESTNET.md	what is deployed on the public testnet, where, and how to check it without trusting the document
adr/README.md	all thirty-seven decision records, with a reading order and the open questions collected
m0/versions.md	every pinned version, why each is pinned that way, and the procedure for changing one

A cost-guardrail failure is a prompt to re-measure and re-publish deliberately, not necessarily a defect — but a published figure never moves without the change that owns it.